

Full Stack Web Development Lab

Week 7 – Record

Aim

To develop a Student Course Management System using Node.js and Express.js that manages student information stored in a JSON file and provides it through HTTP GET routes, including routes that use the built-in **os** and **dns** modules.

Description

This lab builds a basic server-side web application using Node.js and Express.js. Student data is stored in a JSON file (students.json) and read with the **fs** module using fs.readFileSync() and JSON.parse(). The Express server (index.js) runs on port 3000 and defines five GET routes: **/students** returns all students; **/students/:id** uses a URL parameter (req.params.id) to return one student, or a "Student not found" message with status 404; **/search?course=Node.js** uses a query parameter (req.query.course) to filter students by course; **/system** uses the built-in **os** module to return the platform, hostname, total memory and free memory; **/dns** uses the built-in **dns** module (dns.lookup) to find the IP address of google.com. A home page (**/**) with embedded CSS shows the students as cards, with a course search box and buttons for the system and DNS info, using fetch() to call the routes. NPM is used to install Express (npm install express). This demonstrates NPM, built-in modules, file-system operations, Express routing, HTTP methods and URL/query parameters.

Program

index.js (Express Server)

```
const express = require('express');
const fs = require('fs');
const os = require('os');
const dns = require('dns');

const app = express();
const getStudents = () => JSON.parse(fs.readFileSync('students.json', 'utf-8'));

// Home page with CSS (kept inside index.js, no extra files)
const page = `
<!DOCTYPE html>
<html>
<head>
  <title>Student Course Management</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f0f2f5; margin: 0; }
    header { background: #4f46e5; color: white; text-align: center; padding: 25px; }
    .box { max-width: 900px; margin: 25px auto; padding: 0 15px; }
    input, button { padding: 10px; font-size: 15px; border-radius: 6px; border: 1px solid #ccc; }
    button { background: #4f46e5; color: white; border: none; cursor: pointer; }
    .cards { display: grid; grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); gap: 15px; margin-top: 20px; }
    .card { background: white; padding: 18px; border-radius: 10px; box-shadow: 0 2px 6px rgba(0,0,0,0.1); }
    .card h3 { margin: 0 0 8px 0; color: #4f46e5; }
    .card p { margin: 4px 0; color: #555; }
    pre { background: white; padding: 15px; border-radius: 10px; }
  </style>
</head>
<body>
  <header><h1>Student Course Management</h1></header>
  <div class="box">
    <input id="course" placeholder="Search by course (e.g. Node.js)">
    <button onclick="load('/search?course=' + document.getElementById('course').value)">Search</button>
    <button onclick="load('/students')">Show All</button>
    <button onclick="info('/system')">System</button>
    <button onclick="info('/dns')">DNS</button>
    <div class="cards" id="out"></div>
  </div>
<script>
  async function load(url) {
    const data = await (await fetch(url)).json();
```

```

        document.getElementById('out').innerHTML = data.map(s =>
            '<div class="card"><h3>' + s.name + '</h3><p>ID: ' + s.id + '</p><p>Course: ' + s.course + '</p><p>Year: '
            + s.year + '</p></div>'
        ).join('');
    }
    async function info(url) {
        const data = await (await fetch(url)).json();
        document.getElementById('out').innerHTML = '<pre>' + JSON.stringify(data, null, 2) + '</pre>';
    }
    load('/students');
</script>
</body>
</html>`;

app.get('/', (req, res) => res.send(page));

app.get('/students', (req, res) => res.json(getStudents()));

app.get('/students/:id', (req, res) => {
    const s = getStudents().find(s => s.id == req.params.id);
    s ? res.json(s) : res.status(404).send('Student not found');
});

app.get('/search', (req, res) => {
    res.json(getStudents().filter(s => s.course === req.query.course));
});

app.get('/system', (req, res) => {
    res.json({ platform: os.platform(), hostname: os.hostname(), totalMemory: os.totalmem(), freeMemory: os.freemem() });
});

app.get('/dns', (req, res) => {
    dns.lookup('google.com', (err, address) => res.json({ host: 'google.com', address }));
});

app.listen(3000, () => console.log('Server running at http://localhost:3000'));

```

students.json (Student Data)

```

[
  { "id": 1, "name": "Ravi", "course": "Node.js", "year": 3 },
  { "id": 2, "name": "Anil", "course": "React", "year": 2 },
  { "id": 3, "name": "Sneha", "course": "Node.js", "year": 3 },
  { "id": 4, "name": "Priya", "course": "MongoDB", "year": 4 },
  { "id": 5, "name": "Kiran", "course": "Express.js", "year": 2 }
]

```

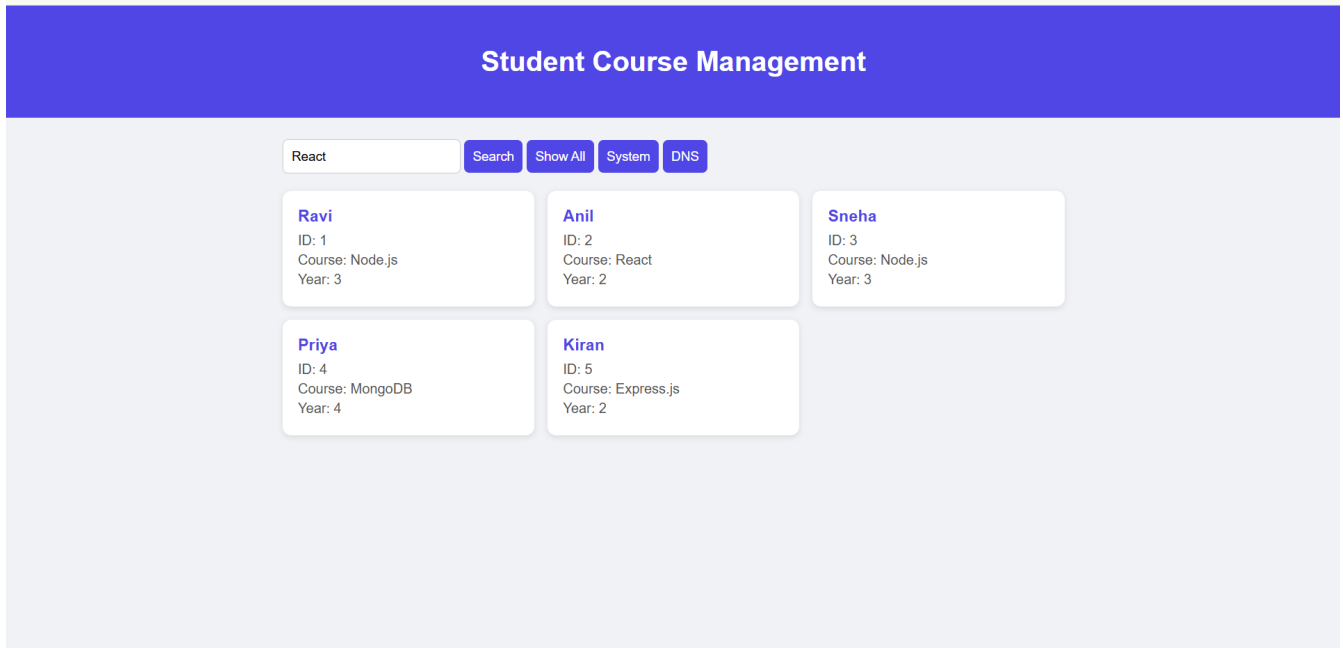
Details about the Program

Where Saved	Both files are saved inside the project folder FL_LAB_ANITS/fs_week_7/ in the GitHub repository raviteja-codehub/FL_LAB_ANITS — index.js is the Express server (with the embedded home page CSS) and students.json holds the student data. node_modules is not uploaded.
Where Executed	The server runs on Node.js from the terminal on the local computer, and the routes are opened in a web browser (Google Chrome / Edge / Firefox) at http://localhost:3000. GitHub Pages cannot run Node.js, so this week is not hosted online.
How Executed	1. Open a terminal in the project folder. 2. Run npm init -y and then npm install express. 3. Start the server with node index.js; the console prints "Server running at http://localhost:3000". 4. Open http://localhost:3000/, /students, /students/1, /search?course=Node.js, /system and /dns in the browser to see the output of each route.

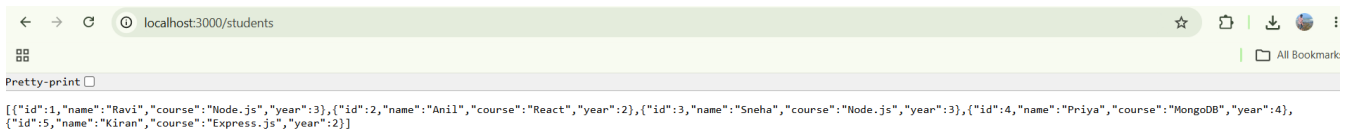
Output

After starting the server with node index.js and opening the routes in the browser, the output is as shown below:

GET http://localhost:3000/



GET http://localhost:3000/students



GET http://localhost:3000/students/1

```
{"id":1,"name":"Ravi","course":"Node.js","year":3}
```

GET http://localhost:3000/search?course=Node.js

```
[{"id":1,"name":"Ravi","course":"Node.js","year":3}, {"id":3,"name":"Sneha","course":"Node.js","year":3}]
```